

Lecture 12 Crash Study Guide

I/O Management and Disk Scheduling

Emergency goal

Understand the pipeline: I/O device differences -> how the CPU performs I/O -> why buffering exists -> disk access-time formulas -> disk scheduling algorithms -> RAID/cache/OS case studies. This guide is written for last-minute exam use: read the high-yield parts first, then practice the formulas and policy comparisons.

0. What to study first if time is short

Time	Focus	What you must be able to say
First 25 minutes	Definitions and comparisons	I/O devices, programmed vs interrupt-driven vs DMA, buffering types, disk performance terms.
Next 35 minutes	Numerical formulas	Access time, transfer time, total average access time, single-buffer timing, disk scheduling head movement.
Next 40 minutes	Disk scheduling + RAID	FIFO, priority, LIFO, SSTF, SCAN, C-SCAN, N-step-SCAN, FSCAN, RAID 0-6.
Last 20 minutes	OS case studies + traps	Disk cache LRU/LFU, UNIX SVR4 I/O, Linux elevator/deadline/anticipatory, Windows I/O manager and async I/O.

Absolute must-know sentence

I/O management is about hiding device differences, improving efficiency, and especially reducing disk delays. Disk performance is dominated by seek time and rotational delay, so buffering, caching, DMA, and disk scheduling are all attempts to reduce waiting and CPU waste.

1. Big picture: what this lecture is about

- **I/O devices differ a lot:** They differ in speed, control complexity, transfer unit, data format, application use, and error behavior.
- **The CPU should not waste time waiting:** Programmed I/O wastes CPU through busy waiting. Interrupts and DMA reduce that waste.
- **Buffering overlaps work:** The OS stores data in system memory so I/O and computation can overlap and swapping problems are reduced.
- **Disk I/O is special:** Mechanical disks are slow because the head must move to a track and wait for rotation.
- **Scheduling improves disk use:** The order of disk requests changes total head movement and therefore performance.
- **RAID improves performance/reliability:** Multiple disks can act as one logical disk, with striping and/or redundancy.

2. I/O devices: categories and differences

Category	Examples	Exam point
Human interaction	Keyboard, mouse, printer, terminal	Usually slower, user-facing, often stream-oriented.

Machine readable	Disks, USB keys, sensors, controllers, actuators	Often block-oriented or device-specific; used by OS for storage and virtual memory.
Communication	Network cards, modems	Data transmission, protocols, errors, buffering.

- **Data rate:** Devices differ by several orders of magnitude. A keyboard is tiny compared with disks/network/graphics.
- **Application:** The device use affects OS policy. A disk stores files, databases, and virtual-memory pages.
- **Control complexity:** A printer is simpler than a disk.
- **Unit of transfer:** Terminals often transfer byte streams; disks transfer blocks.
- **Data representation:** Different encodings and parity conventions.
- **Error conditions:** Different devices report errors differently, and the OS response differs.

Professor trap

Do not say all I/O devices are handled the same internally. The OS tries to give a uniform interface, but the lower layers must handle device-specific speed, transfer unit, control, and errors.

3. Techniques for performing I/O

Technique	How it works	High-yield exam sentence
Programmed I/O	CPU issues I/O command and then busy-waits until done.	Simple but wastes CPU. The process keeps checking the device.
Interrupt-driven I/O	CPU issues I/O command. Device interrupts CPU when done.	CPU can do other work. Blocking requests put the process in blocked state.
DMA	DMA module transfers a block directly between memory and device.	CPU is involved at start/end only; interrupted after whole block transfers. Best for blocks.

- **Blocking I/O request:** The OS places the process in blocked state and schedules another process.
- **Non-blocking I/O request:** The issuing process can continue while I/O is happening.
- **DMA advantage:** Much less CPU involvement for large transfers.

4. Evolution of the I/O function

Stage	Idea	Why it matters
1	CPU directly controls peripheral	Primitive; CPU handles everything.
2	I/O module/controller added	CPU no longer handles details, but may still use programmed I/O.
3	Controller with interrupts	CPU does not waste time waiting.
4	DMA	Blocks moved without CPU involvement except start/end.
5	I/O module becomes separate processor	CPU gives it an I/O program to execute.
6	I/O processor with local memory	Common for complex terminal/communication control.

5. DMA: what the CPU sends and DMA configurations

- **CPU sends to DMA:** read/write request, device address, starting memory location, and number of words/bytes to transfer.
- **DMA stores:** memory starting address in its address register and count in its data count register.

- **Then CPU continues:** DMA moves data directly between memory and device over the bus.
- **Completion:** DMA sends an interrupt to the processor.

DMA configuration	Structure	Exam point
Single bus, detached DMA	Processor, DMA, I/O, and memory share one bus.	Inexpensive but inefficient. Transfer may consume extra bus cycles.
Integrated DMA and I/O	DMA is integrated with I/O function; direct path between DMA and I/O.	Cuts required bus cycles.
I/O bus configuration	I/O modules connect to DMA over separate I/O bus.	Reduces DMA interfaces; system bus mainly used for DMA-memory exchange and CPU control signals.

6. OS design issues: efficiency and generality

- **Efficiency:** Most I/O devices are slow compared with memory and CPU. Multiprogramming helps because one process can run while another waits for I/O.
- **Swapping issue:** Swapping helps bring ready processes in, but swapping itself is an I/O operation.
- **Disk I/O focus:** A major I/O design goal is improving disk I/O efficiency.
- **Generality:** The OS wants a uniform interface: read, write, open, close, lock, unlock.
- **Layering:** Details of device-specific I/O are hidden in lower-level routines and drivers.

7. I/O buffering

- **Definition:** Input transfers can be done in advance of requests, and output transfers can be done after the user request.
- **Main reason:** Overlap I/O with processing to increase efficiency.
- **Swapping/deadlock reason:** If no buffering is used, pages involved in I/O must stay locked in memory. If the process is swapped out while waiting for I/O, the process waits for I/O and I/O waits for the process to be swapped in.

Type	Meaning	Examples
Block-oriented buffering	Fixed-size blocks; transfers one block at a time; blocks can be referenced by block number.	Disks and USB keys.
Stream-oriented buffering	Stream of bytes instead of blocks.	Terminals, printers, communication ports, mouse, sensors/controllers.

7.1 No buffer, single buffer, double buffer, circular buffer

Buffering method	Meaning	High-yield note
No buffer	OS directly accesses the device when needed.	Simple but no overlap; process waits.
Single buffer	OS assigns one system buffer in main memory. Input goes to system buffer, then moves to user space. Next block can be read ahead.	Read ahead / anticipated input. Good when sequential access is likely.
Double buffer	Two system buffers. Process uses one while OS fills/empties the other.	Better overlap than single buffer.
Circular buffer	More than two buffers arranged as a circular buffer.	Useful for rapid bursts of I/O; bounded-buffer producer/consumer problem.

Buffer formulas

No buffer: Execution time = $T + C$

Single buffer: Execution time = $\max(C, T) + M$

T = disk I/O time for one block, C = computation time between requests, M = time to move data into user space.

Double buffer: if $(C + M) < T$, the I/O device works at full speed. If $(C + M) > T$, the process will not wait for I/O; more buffers can improve efficiency.

8. Stream-oriented single-buffer I/O

Mode	Meaning
Line-at-a-time I/O	Used on scroll-mode terminals and line printers. Input line ends with carriage return. Buffer holds one line. User process waits during input until the whole line arrives. Output can be placed in buffer and process continues unless second line arrives before first output finishes.
Byte-at-a-time I/O	Used when every keystroke matters, such as forms-mode terminals, sensors, controllers. Interaction follows producer/consumer model.

9. Disk components and performance

- **Disk components to recognize:** platter, spindle, track, sector, cylinder, arm, arm assembly, read/write head, rotation.
- **To read/write:** the head must be positioned at the correct track and sector.

Disk performance formulas

Access time = seek time + rotational delay

Transfer time: $T = b / (rN)$

Total average access time: $T_a = T_s + 1/(2r) + b/(rN)$

where T_s = average seek time, r = rotation speed in revolutions/second, b = bytes transferred, N = bytes per track. Average rotational delay is $1/(2r)$.

- **Seek time:** time to move the head to the desired track.
- **Rotational delay / latency:** time for the start of the sector to rotate under the head.
- **Transfer time:** time to actually move data after the head and sector are positioned.
- **Big exam idea:** The order of reading sectors has a huge effect on I/O performance.

10. Timing comparison example from the lecture

- Disk: average seek time = 4 ms, rotation speed = 7500 rpm = 125 rps, sector size = 512 bytes, 500 sectors per track.
- File: 2500 sectors = 5 tracks = 1.28 MB.

Case	Situation	Result
Sequential organization	File occupies 5 adjacent tracks.	First track = seek 4 ms + avg rotation 4 ms + transfer 8 ms = 16 ms. Next 4 tracks = $4 * (4 + 8) = 48$ ms. Total = 64 ms = 0.064 s.
Random access	2500 sectors are distributed randomly.	Each sector = seek 4 ms + rotation 4 ms + transfer 0.016 ms = 8.016 ms. Total = $2500 * 8.016 = 20,040$ ms = 20.04 s.

Why this example matters

Sequential storage is dramatically faster than random access because it avoids repeated seeks and rotational waits. This is why disk scheduling and file layout matter.

11. Disk scheduling policies

- **Goal:** Reduce average seek time by choosing a better request order.
- **Lecture example setup:** Head starts at track 100. Disk has 200 tracks. Request queue: 55, 58, 39, 18, 90, 160, 150, 38, 184.

Policy	How it chooses	Exam note
FIFO / FCFS	Serve requests in arrival order.	Fairest and simple. Performance can be bad if requests jump around.
Priority	Serve based on process priority/objectives.	Not mainly for disk optimization; can hurt long jobs or database workloads.
LIFO	Serve most recent request first.	Exploits locality and can improve throughput, but starvation possible.
SSTF	Choose request requiring least arm movement from current position.	Better than FIFO, but can starve far requests and does not always globally minimize average seek.
SCAN / Elevator / LOOK	Move arm in one direction satisfying requests, then reverse.	Good service distribution; avoids some starvation; favors edge/latest requests in lecture notes.
C-SCAN	Scan one direction only; after end, return to opposite end and continue.	Reduces maximum delay and service variability; fairer to tracks at periphery.
N-step-SCAN	Split queue into subqueues of size N; process each using SCAN.	Prevents arm stickiness and gives service guarantees.
FSCAN	Uses two queues. New requests go into the other queue while current queue is being scanned.	Defers new arrivals until old batch is done; load sensitive.

11.1 Worked comparison for the lecture request queue

Algorithm	Access order in slide example	Result
FIFO	55, 58, 39, 18, 90, 160, 150, 38, 184	Average seek length 55.3
SSTF	90, 58, 55, 39, 38, 18, 150, 160, 184	Average seek length 27.5
SCAN increasing first	150, 160, 184, 90, 58, 55, 39, 38, 18	Average seek length 27.8
C-SCAN increasing first	150, 160, 184, 18, 38, 39, 55, 58, 90	Average seek length 35.8

How to solve disk scheduling questions

1) Draw a number line. 2) Put the current head position. 3) Mark all requests. 4) Follow the algorithm exactly. 5) Add absolute movement between consecutive tracks. 6) Average seek length = total movement / number of serviced requests.

12. RAID: Redundant Array of Independent Disks

- **Why RAID exists:** Disk performance improves slower than CPU performance, so multiple disks in parallel can increase I/O performance.
- **RAID as OS view:** A set of physical drives appears as one logical drive.
- **Core ideas:** data distributed across drives, parallel I/O, redundancy/parity for recovery.
- **Levels:** RAID 0 through RAID 6.

Level	Main idea	Must remember
RAID 0	Striping only	Very high performance. Not true RAID because no redundancy; disk failure is catastrophic.
RAID 1	Mirroring	Duplicates all data. Reads can use either disk. Writes update both. Good reliability but double cost.
RAID 2	Bit/word striping + Hamming code	Corrects single-bit errors and detects double-bit errors. Costly and rarely useful unless disk errors are high.
RAID 3	Bit-interleaved parity with one parity disk	Parallel access; high transfer rate. Only

		one request at a time, so poor for transaction-heavy workloads.
RAID 4	Block-level parity with dedicated parity disk	Independent disks can serve parallel requests, but parity disk is bottleneck. Write penalty.
RAID 5	Block-level distributed parity	Like RAID 4 but parity distributed round-robin, removing single parity disk bottleneck.
RAID 6	Dual distributed parity	Can tolerate two disk failures. Needs N+2 disks and has substantial write penalty; lecture notes mention ~30% drop over RAID 5.

RAID parity idea

For RAID 3/4/5 parity, XOR is used. Example with data disks X0-X3 and parity disk X4:

$X4(i) = X3(i) \text{ XOR } X2(i) \text{ XOR } X1(i) \text{ XOR } X0(i)$

If X1 fails:

$X1(i) = X3(i) \text{ XOR } X2(i) \text{ XOR } X4(i) \text{ XOR } X0(i)$

RAID 4 write update trick:

new parity = old parity XOR old data strip XOR new data strip.

13. Disk cache

- **Definition:** A disk cache is a buffer in main memory that stores copies of disk sectors.
- **Cache hit:** If requested data is in cache, pass it to user without reading disk.
- **Cache miss:** If not in cache, read from disk and add the sector/block to cache.
- **Why it works:** Locality of reference: once a block is fetched, it is likely to be referenced again soon.

Policy	How it works	Trap
LRU - Least Recently Used	Replace the block that has gone longest without being referenced.	Implemented as a stack/list with most recently referenced block on top. Commonly used.
LFU - Least Frequently Used	Replace block with fewest references.	Uses counters. Weakness: a block used many times briefly may keep a misleading high count.

14. UNIX SVR4 I/O

- **Special files:** Each I/O device is associated with a special file such as /dev/xxx and managed by the file system.
- **Uniform interface:** Ordinary read/write calls access devices.
- **Device categories:** character devices such as terminals/printers; block devices such as disks/tapes.
- **Buffered I/O:** passes through buffer cache and character queue.
- **Unbuffered I/O:** uses DMA directly between I/O module and process area.

SVR4 part	Meaning
Buffer cache	Disk cache. Maintains free list, device list, and driver I/O queue. Uses hash table for fast lookup and LRU ordering.
Character cache/queue	Used by terminals/printers. Producer/consumer model. Data is destroyed as it is read.
Unbuffered I/O	DMA directly between device and process space. Fastest, but process is locked in memory and device is tied to the process during transfer.

15. Linux I/O scheduling

- **Linux and UNIX similarity:** Linux associates a special file with each I/O device driver.
- **Recognized devices:** block, character, and network devices.
- **Older Linux scheduling:** variation of Elevator / LOOK in Linux 2.4.
- **Linux 2.6 schedulers:** Deadline scheduler and anticipatory I/O scheduler.

Scheduler	How it works	Why it matters
Elevator / SCAN / LOOK	Single queue sorted by block number; drive moves in one direction satisfying requests.	Problems: distant blocks can be delayed/starved; writes can starve reads because writes are asynchronous but reads block processes.
Deadline scheduler	Uses sorted elevator queue plus FIFO read queue plus FIFO write queue. Requests have expiration time: lecture says about 0.5 s for reads and 5 s for writes.	Overcomes starvation and read-vs-write problem.
Anticipatory scheduler	After satisfying a read, waits briefly (about 6 ms) to see whether same process issues a nearby read.	Uses locality. If nearby read appears, service it immediately; otherwise continue deadline scheduling.

16. Windows I/O

- **I/O manager:** Responsible for all OS I/O and provides a uniform interface for drivers.
- **Works with:** cache manager, file system drivers, network drivers, hardware device drivers.
- **Cache manager:** handles file caching for all file systems. Updates may be recorded only in cache; lazy writer periodically batches writes to disk.
- **File system drivers:** I/O manager treats file system drivers as ordinary device drivers and routes volume requests to the proper driver.
- **Hardware drivers:** access hardware registers using the Hardware Abstraction Layer.

Mode	Meaning
Synchronous I/O	Application blocks until I/O finishes.
Asynchronous I/O	Application continues and is informed later. Used whenever possible for performance.

- **Windows completion signals:** signal the file object, signal an event kernel object, Asynchronous Procedure Call (APC), I/O completion ports.
- **I/O completion ports:** use a pool of threads to handle requests.
- **Windows RAID support:** hardware RAID via disk controller; software RAID in OS. Lecture mentions RAID 0, RAID 1, and RAID 5 support.

17. Visuals and diagrams you should be able to explain

Slide/visual	What to say if it appears in exam
Page 3: data-rate chart	Explain that device speeds vary by orders of magnitude, so OS policies cannot treat all devices identically internally.
Pages 7-9: DMA configurations	Know single bus vs integrated DMA/I/O vs I/O bus and why bus cycles matter.
Pages 13-17: buffering diagrams	Know no buffer, single, double, circular buffer and how they overlap I/O with computation.
Page 19: disk components	Recognize platter, track, sector, cylinder, read/write head, spindle, arm.
Page 29: scheduling comparison	Given a start track and request queue, compute movement and average seek length.
Page 42: RAID comparison	Compare RAID levels by redundancy, disks required, transfer capacity, and small I/O performance.

18. Likely exam questions and model answers

Q1. Compare programmed I/O, interrupt-driven I/O, and DMA.

Programmed I/O makes the CPU issue a command and busy-wait. Interrupt-driven I/O lets the CPU continue or schedule another process until the device interrupts. DMA transfers blocks directly between memory and device; the CPU is interrupted only when the whole block is done.

Q2. Why is buffering needed?

To overlap I/O with computation, improve efficiency, and avoid problems with swapping. Without buffering, I/O pages must remain locked; swapping a blocked process can create a deadlock where the process waits for I/O and the I/O waits for the process to be swapped back in.

Q3. Difference between block-oriented and stream-oriented buffering?

Block-oriented I/O stores fixed-size blocks and transfers one block at a time, used for disks and USB. Stream-oriented transfers bytes, used for terminals, printers, communication ports, mouse, and sensors.

Q4. Explain seek time, rotational delay, and transfer time.

Seek time moves the head to the right track. Rotational delay waits for the sector to reach the head. Transfer time moves data as the sector passes under the head.

Q5. Why is sequential disk access much faster than random access?

Sequential access avoids repeated seeks and rotational delays. In the lecture example, reading 2500 sequential sectors takes 64 ms, while random access takes about 20.04 s.

Q6. Compare FIFO, SSTF, SCAN, and C-SCAN.

FIFO is fair but may have poor seek performance. SSTF chooses closest request and improves average seek but may starve distant requests. SCAN moves like an elevator and gives better distribution. C-SCAN scans one direction and returns quickly, reducing maximum delay.

Q7. What is arm stickiness?

It happens when the disk arm stays near one track/area because many requests keep arriving there, possibly monopolizing the device. N-step-SCAN and FSCAN reduce this by batching requests.

Q8. Why is RAID 0 not true RAID?

It stripes data for performance but has no redundancy; one disk failure can destroy the array.

Q9. RAID 4 vs RAID 5?

RAID 4 uses a dedicated parity disk, causing a parity bottleneck. RAID 5 distributes parity across all drives, removing that single bottleneck.

Q10. Why can writes starve reads in an elevator scheduler?

Writes are often asynchronous: the kernel copies data into buffer and the process continues. Reads are synchronous: the process waits for data. Many large writes can delay reads, hurting responsiveness.

19. High-yield traps

- DMA does not mean the CPU is never involved. The CPU starts the transfer and receives an interrupt at the end.
- Access time is not the same as transfer time. Access time gets the head/sector ready; transfer time actually moves data.
- Average rotational delay is half a rotation: $1/(2r)$, not $1/r$.
- Best average seek in a fixed example does not mean universally best scheduler. SSTF can starve far requests.
- SCAN and LOOK are treated together in the lecture: elevator movement in one direction, then reverse.
- C-SCAN reduces maximum delay by scanning only one direction, then returning to the other end.
- Single buffering still has M, the move to user space. Do not forget the +M.
- Circular buffer is basically the bounded-buffer producer/consumer problem from concurrency.
- RAID 1 improves reliability but doubles storage cost.
- RAID 3 has high transfer rate but low request rate because all disks participate in each request.
- Unbuffered I/O is fastest but locks the process in memory and ties the device to that process.

20. Flashcards for the last 15 minutes

Card	Card
Programmed I/O? CPU issues command and busy-waits.	Interrupt-driven I/O? Device interrupts when done; CPU can do other work.
DMA? Block transfer directly between memory and device; CPU involved at start/end.	Block-oriented buffering? Fixed-size blocks, used for disks/USB.
Stream-oriented buffering? Bytes/lines, used for terminals/printers/mouse/ports.	No buffer time? $T + C$.
Single buffer time? $\max(C, T) + M$.	Access time? seek time + rotational delay.
Transfer time? $b/(rN)$.	Average rotational delay? $1/(2r)$.
FIFO disk scheduling? Serve in arrival order; fair but can perform badly.	SSTF? Choose nearest request; good average, possible starvation.
SCAN? Elevator: one direction then reverse.	C-SCAN? One direction only, then return to opposite end.
N-step-SCAN? Process queue in batches of N using SCAN.	FSCAN? Two queues; new requests wait in other queue.
RAID 0? Striping, no redundancy.	RAID 1? Mirroring.
RAID 4? Block parity on one dedicated parity disk.	RAID 5? Distributed parity.
RAID 6? Dual distributed parity, tolerates two disk failures.	LRU cache? Replace least recently used block.
LFU cache? Replace least frequently used block.	Deadline scheduler? Sorted queue plus read/write FIFO expiry queues.
Anticipatory scheduler? Wait briefly after read for nearby next read.	
Final checklist before entering the exam I can explain why I/O is slow and why OS hides device differences. I can compare programmed I/O, interrupts, and DMA. I can use buffering formulas and explain no/single/double/circular buffer. I can define seek time, rotational delay, transfer time, and total access time. I can solve FIFO/SSTF/SCAN/C-SCAN head movement questions. I can compare RAID 0-6 and explain parity. I can explain disk cache LRU/LFU. I can summarize UNIX/Linux/Windows I/O case studies.	